Machine Programming

Lecture 16 – PL Kaleidoscope II: Formal Program Verification

Ziyang Li

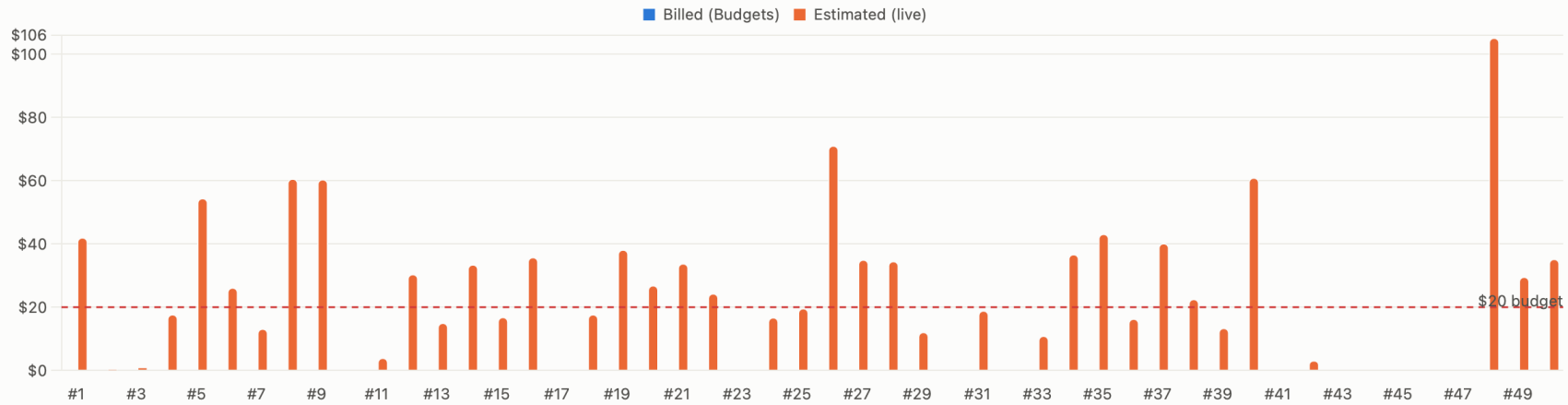
Logistics

- Oral presentation schedule:
  - Releasing in the afternoon
- Assignment 3:
  - Deadline extended to Thursday
  - For those of you who cannot access your AWS account: try check your AWS_REGION and whether that is set to the same region as your reg.
- Final project Group Formation
  - Deadline this Thursday

$0.00	$1138.1	111.0k	37 / 48	0	$42.1
Billed to students (Budgets)	Estimated, live (tokens)	Requests	Students active	Cut off by budget	Instructor keys (1–2)

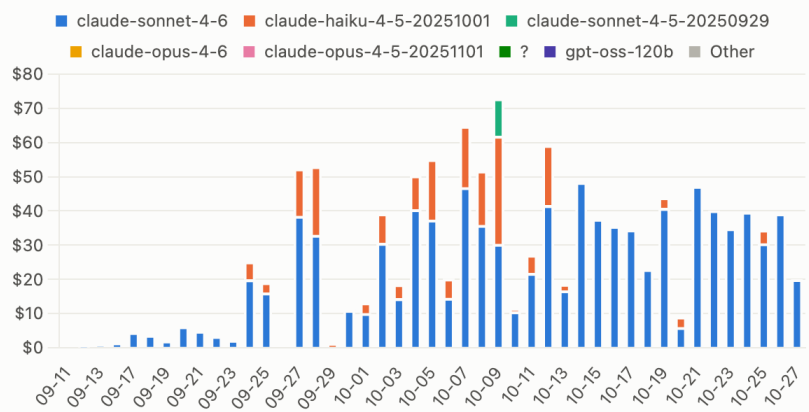
Spend per student vs. $20 budget

Billed = AWS Budgets actual spend (exact, ~24h lag). Estimated = live token counts × list price (minutes lag).



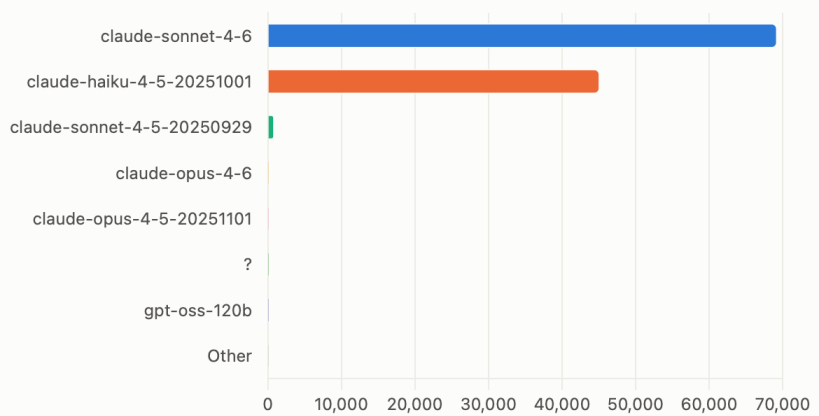
Daily estimated cost by model

All students, from invocation logs.

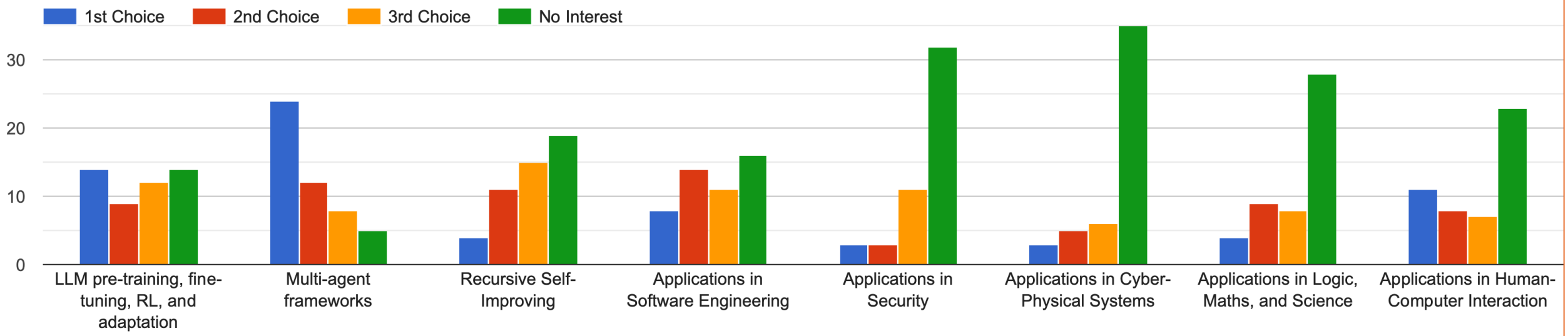


Requests by model

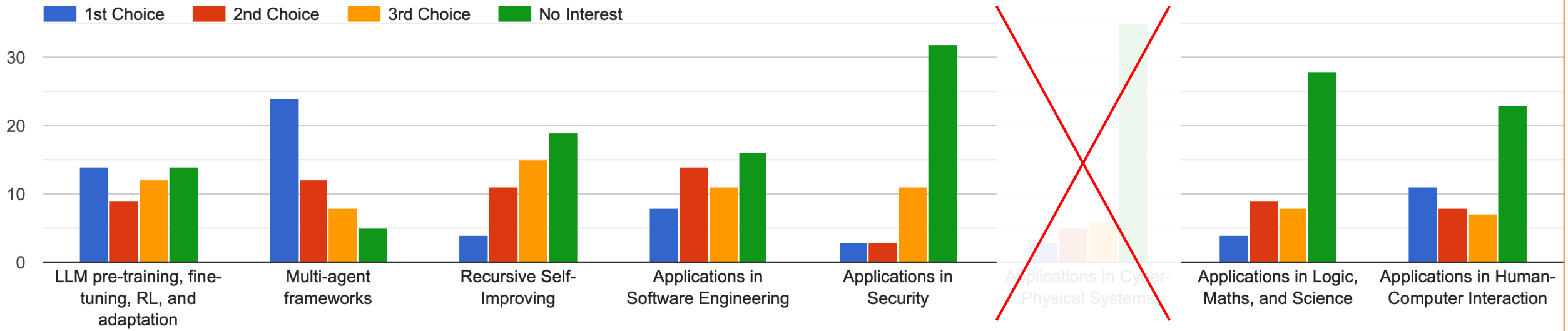
All students, semester to date.



Topic Preferences



Topic Preferences

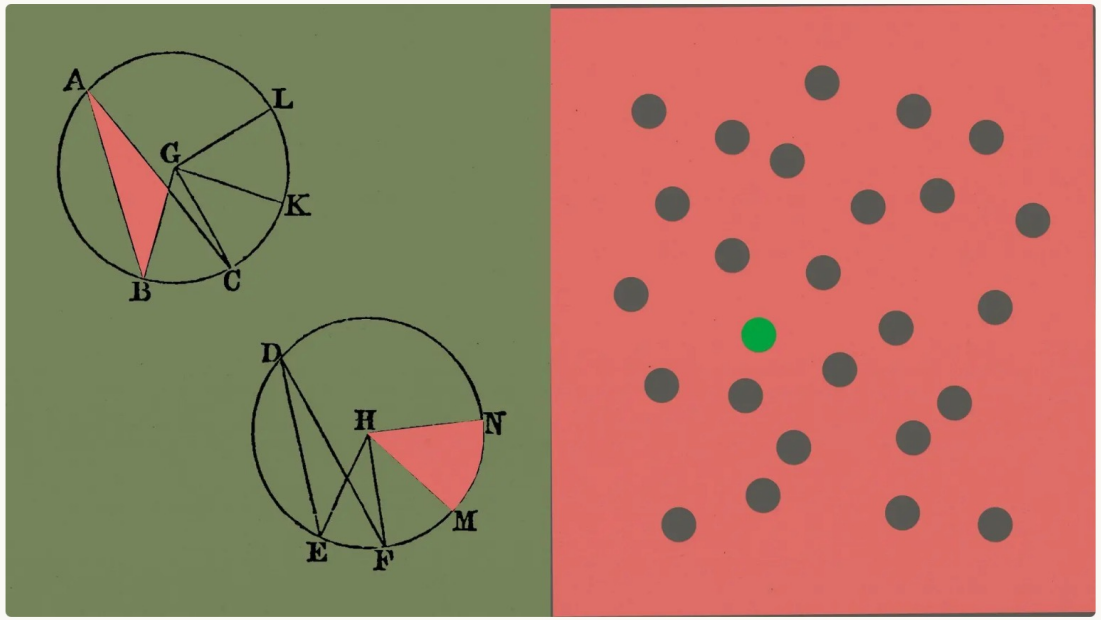


Today’s News

ments

ANTHROP\C

LLMs are making formal software verification feasible at a much larger scale

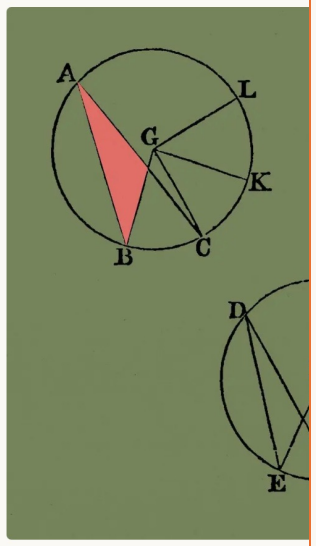


Published	Oct 22, 2026
Team	Frontier Red Team
Topics	Cybersecurity Coding Open source See more...

ments

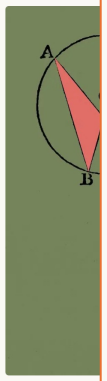
ANTHROP\C

LLMs are making formal software verification feasible at a much larger scale



Most software is riddled with bugs, which AI can now easily find and exploit. The standard defenses against cyberattacks are no longer sufficient: we need ways to prove that code is free of bugs. Formal software verification could provide this guarantee, but it has historically required years of expert effort, making it impractical for most real-world projects. In a collaboration with Theorem, however, we show this is no longer the case: LLMs can now produce formal proofs at large scale, enabling new software development workflows that include proven security guarantees. Using Claude, we proved security properties at the assembly code level (RISC-V) in over 100 real-world open-source software libraries. ***To our knowledge, this represents the first large-scale effort to use AI to apply formal verification to production software at the assembly code level.*** *While writing the proofs, Claude also discovered bugs that had eluded detection for years, despite trillions of test runs. As model capabilities continue to improve, we expect formal verification to become an essential part of secure software development.*

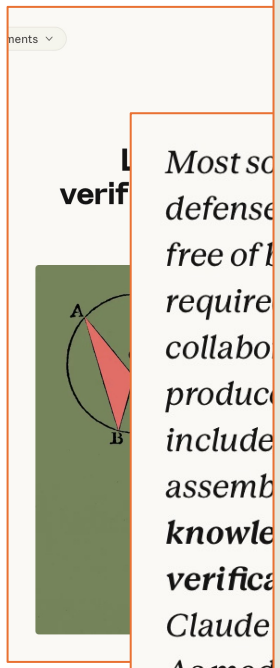
ANTHROP\C



Most software is riddled with bugs, which AI can now easily find and exploit. The standard defenses against cyberattacks are no longer sufficient: we need ways to prove that code is

But LLMs are making it far easier. Frontier models can now produce correctness proofs for real-world software fully autonomously. To demonstrate this, we asked Claude to prove useful security properties across 123 open-source libraries, including showing that some of their executable artifacts can *never* crash under certain conditions. Specifically, Claude proved that these programs will always exit normally without triggering memory-safety errors or failing assertions, for every input less than 64 MB and every modeled Linux operating system (OS) response and memory layout, under certain assumptions (namely that the program runs as a single-threaded static binary with a fixed memory budget on a RISC-V [1] CPU using Linux OS). This is still far away from fixing *all* possible bugs, but it does get us much closer than before and provides a roadmap toward that end.

We're publishing a subset of these proofs so that anyone can verify and reuse them, along with the RISC-V and OS semantics. Theorem, our collaborator on this project, is also offering to run the same pipeline for other open-source projects, and is publishing a Claude skill to help developers get started with verifying their own code.

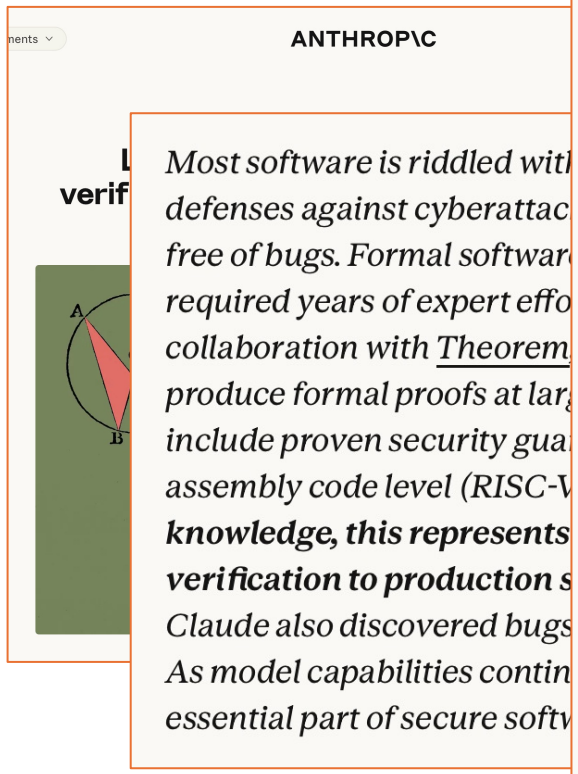


Most so
defense
free of k
require
collabo
produc
include
assemb
knowle
verifica
Claude
As mod
essenti

From code to a machine-checked guarantee

Trusted: assumed correct; could have bugs

Proven: true, if the trusted parts are correct



Limitations

In practice, formal verification can sometimes prove less about the security of the code than we would like.

"Formally verified" does not mean "bug-free." Claude can write a proof showing that a compiled program's behavior satisfies the requirements of a formal spec (which is verified by an external proof checker). However, a proof checker can't tell you whether your spec matches reality. To match reality, the spec must correctly encompass and model everything about the real-world machine that the verified program runs on, including the CPU and operating system. When bugs show up in verified code, it's because the spec, or the assumed semantics of the machine, differs from reality. Fortunately, this also means we know *where* to look for remaining bugs, and how to design a system where those still-possibly-buggy gaps are minimized. Below are the ways this applies to our proofs, and ideas for future work that would close those gaps.

The real machine does something the model doesn't describe. Our proofs are about single-threaded, static RISC-V binaries linked against musl libc. However, most people run these libraries on x86 or ARM and dynamically link them against glibc. To close this gap, we'd need system models that more closely resemble deployed systems. For example, a step toward this would be generating accurate semantics for x86.

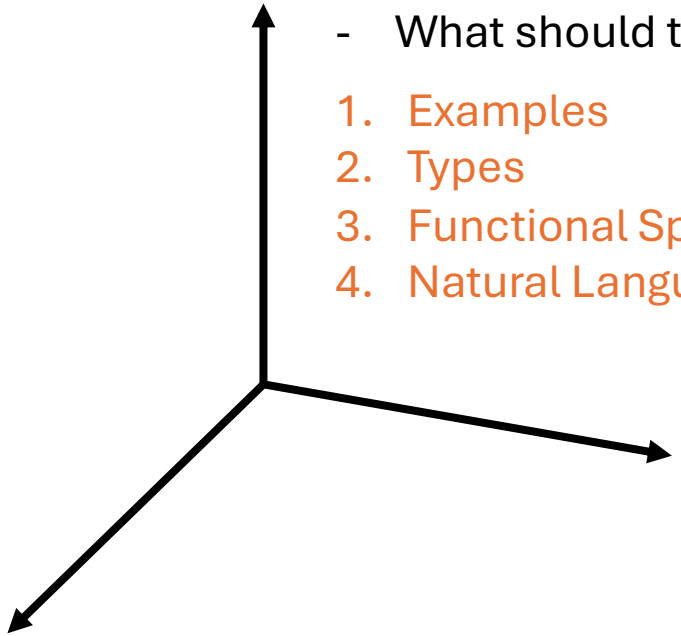
The claim may not cover everything we care about. We proved that a crash never triggers for any input under 64 MB on many fuzzing entry points in Google OSS-Fuzz, including ASan crashes. However, ASan doesn't detect all memory safety bugs, and we didn't verify all the entry points for each library. Even if we had, the whole library wouldn't be covered by verifying all entry points (especially if there is dead code). Additionally, real programs might misuse the library, leading to memory corruption in the *caller*. To close that gap, we'd need to specify what callers must do for each public function, and callers would have to strictly adhere to it.

We expect that as we learn more about leveraging LLMs for formal methods, and as models continue to improve, these limitations will become tractable, and we'll be able to verify a much larger percentage of the code we rely upon.

The Course So Far

Behavioral Specification
- What should the program do?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language



Synthesis Strategy
- How do we find such a program?

Language Models
- Pre-training, Post-training, etc.
- Prompting
- Iterative refinement
- Agentic frameworks

Structural Specification
- What is the space of the programs?

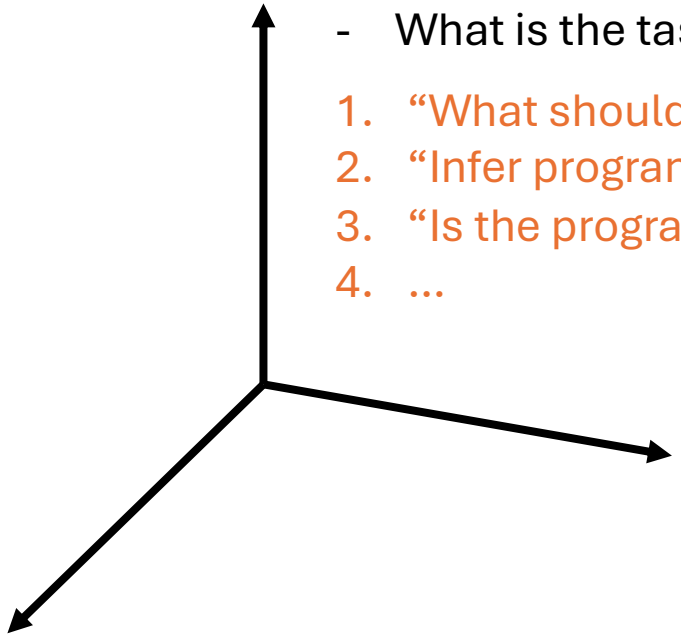
General Purpose Programming Language
Python / Java / C / Rust / …

Domain Specific Languages

The discussion today

Behavioral Specification
- What should the program do?
- What is the task for producing this program?

1. “What should program do”: **Specification Synthesis**
2. “Infer program invariants”: **Specification Inference**
3. “Is the program correct”: **Program Verification**
4. …



Synthesis Strategy
- How do we find such a program?

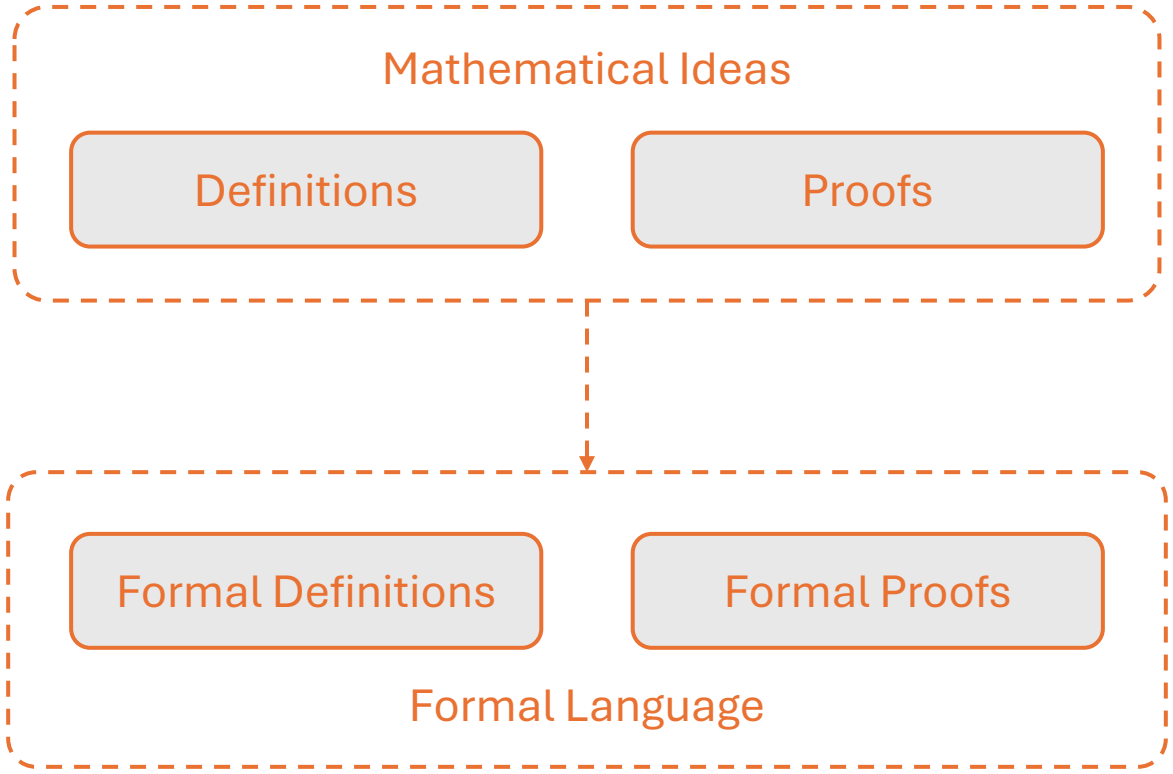
Language Models
- Pre-training, Post-training, etc.
- Prompting
- Iterative refinement
- Agentic frameworks

Structural Specification
- What is the space of the programs?

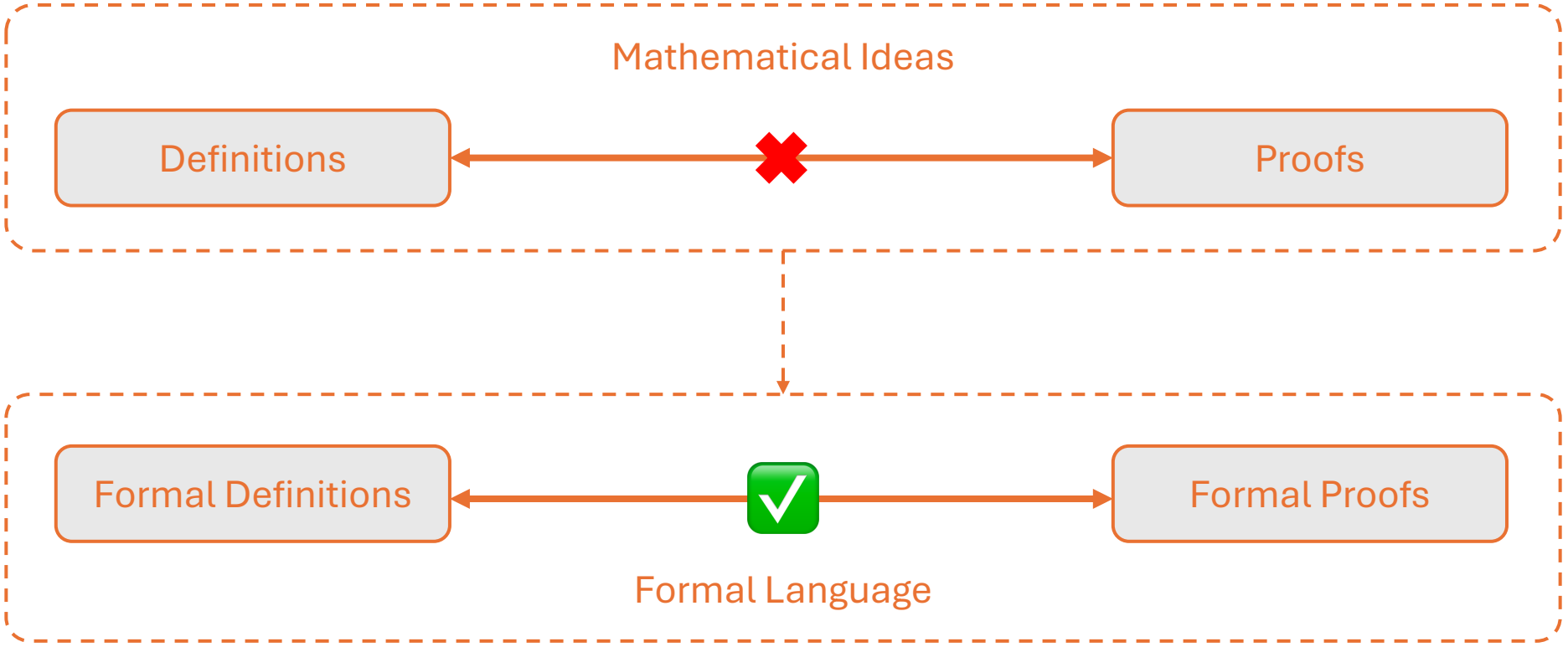
General Purpose Programming Language
Python / Java / C / Rust / …

Domain Specific Languages
Lean/Rocq/Isabelle/Z3 for theorem proving

Central Thesis



Central Thesis



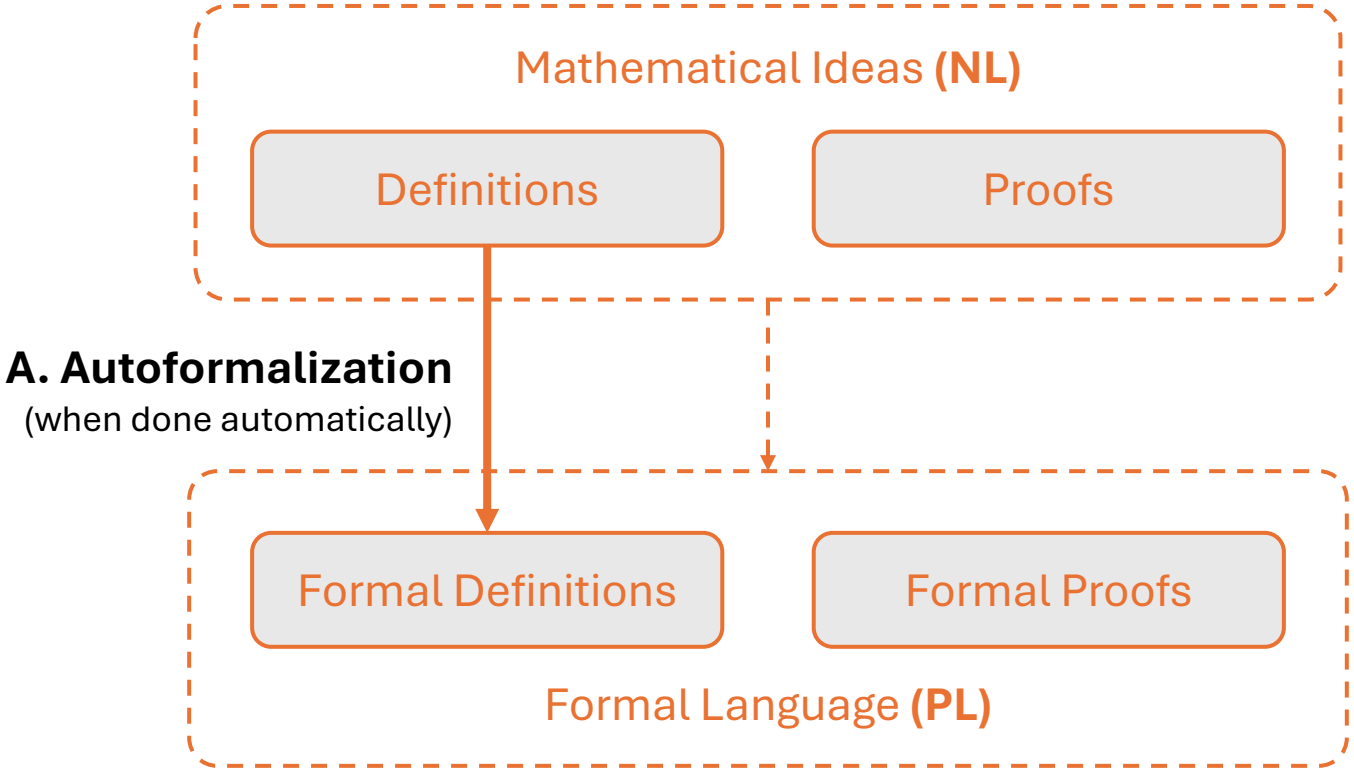
Central Thesis

1. Informal mathematics can be written as a statement in a formal language intended to carry the same mathematical meaning.
   - a statement in a theorem prover is a type

2. A theorem prover is that language plus a small kernel that mechanically checks the proof. If it compiles, the proof is valid.
   - relative to the axioms, the kernel, and the statement you actually wrote

Formal Verification

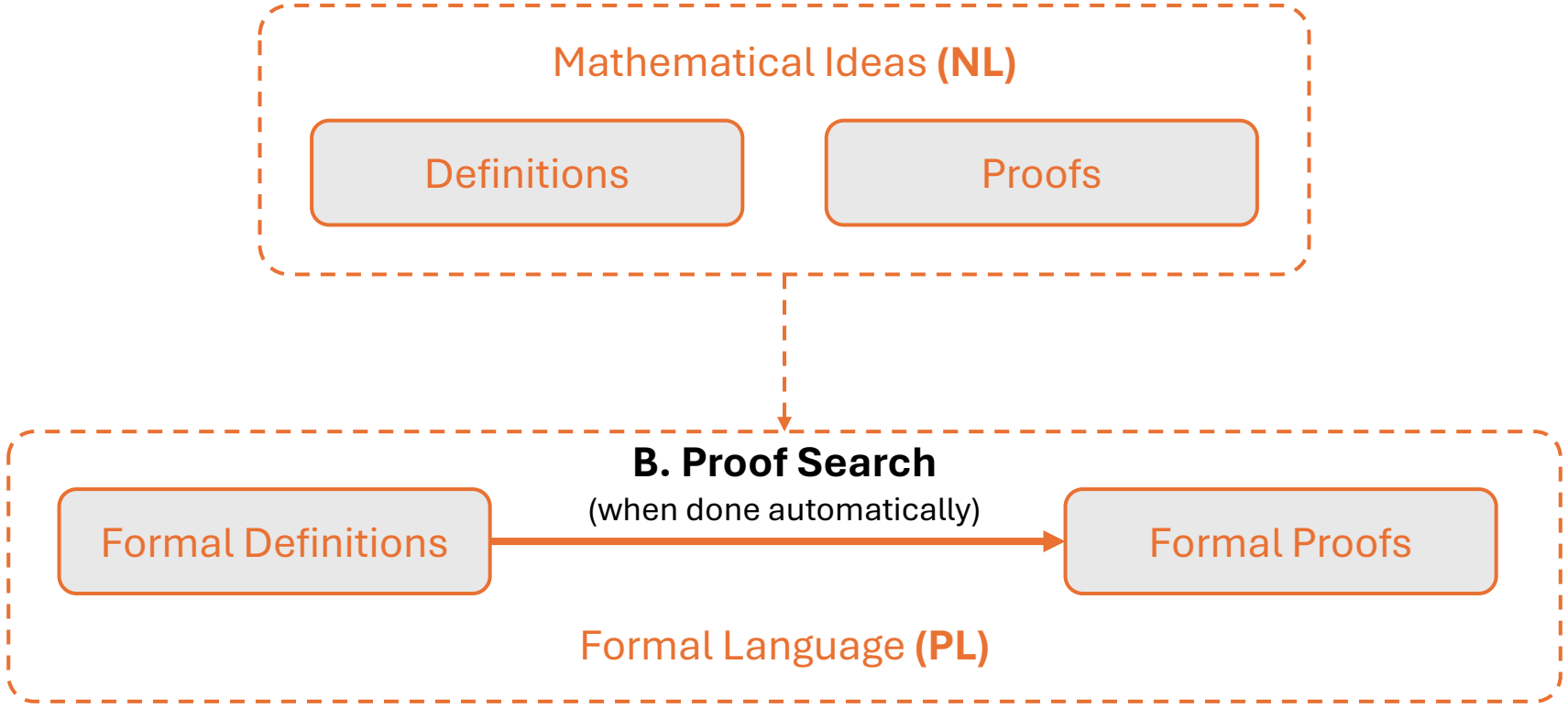
Problem Definition for Machine Programming



Natural Language (NL)

Programming Language (PL)

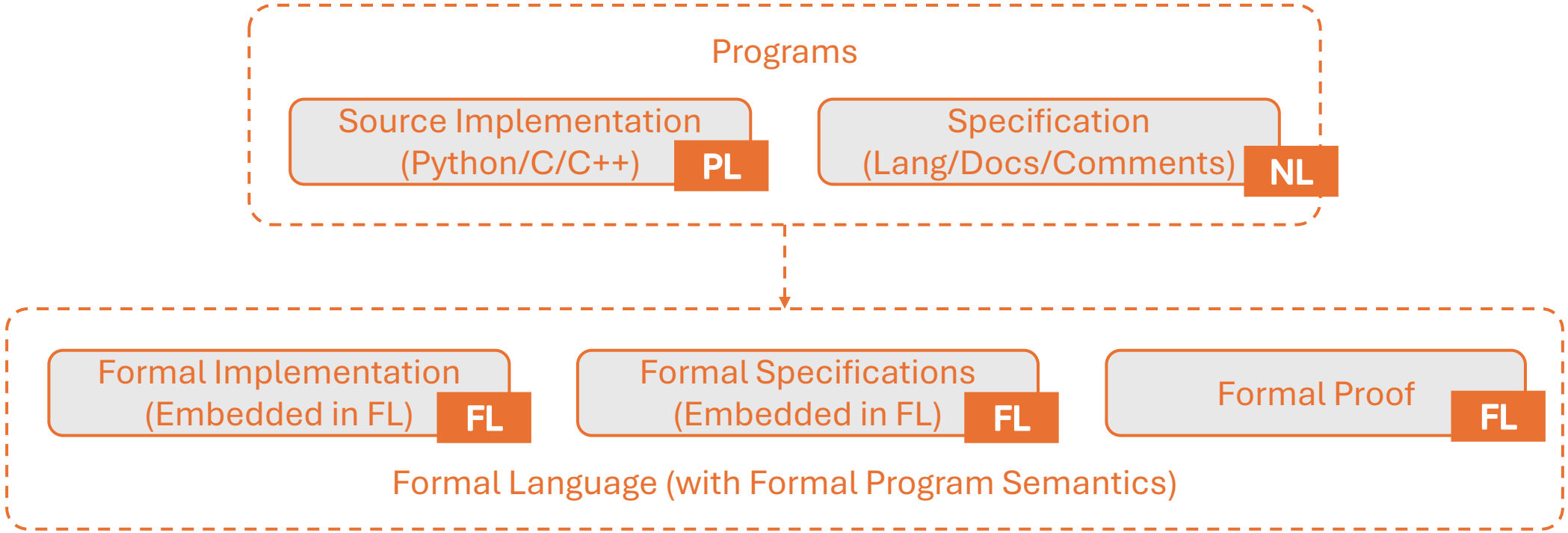
Problem Definition for Machine Programming



Natural Language (NL)

Programming Language (PL)

Formal Program Verification

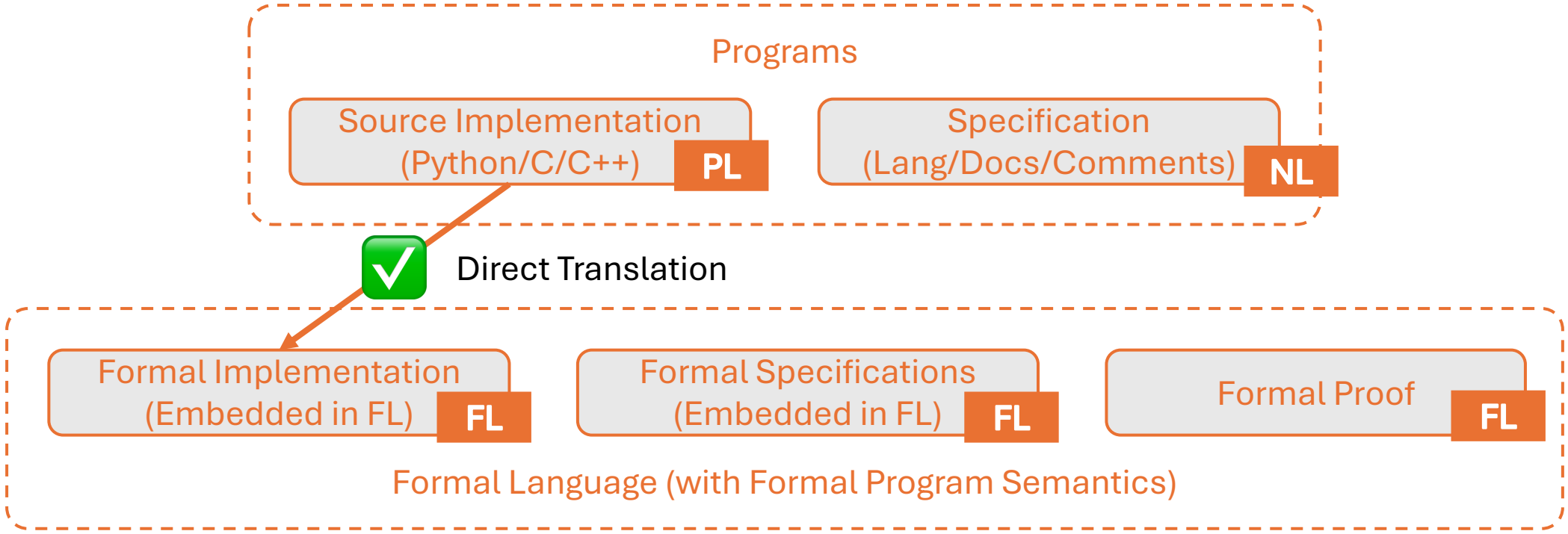


Natural Language (NL)

Programming Language (PL)

Formal Language (FL)

Formal Program Verification

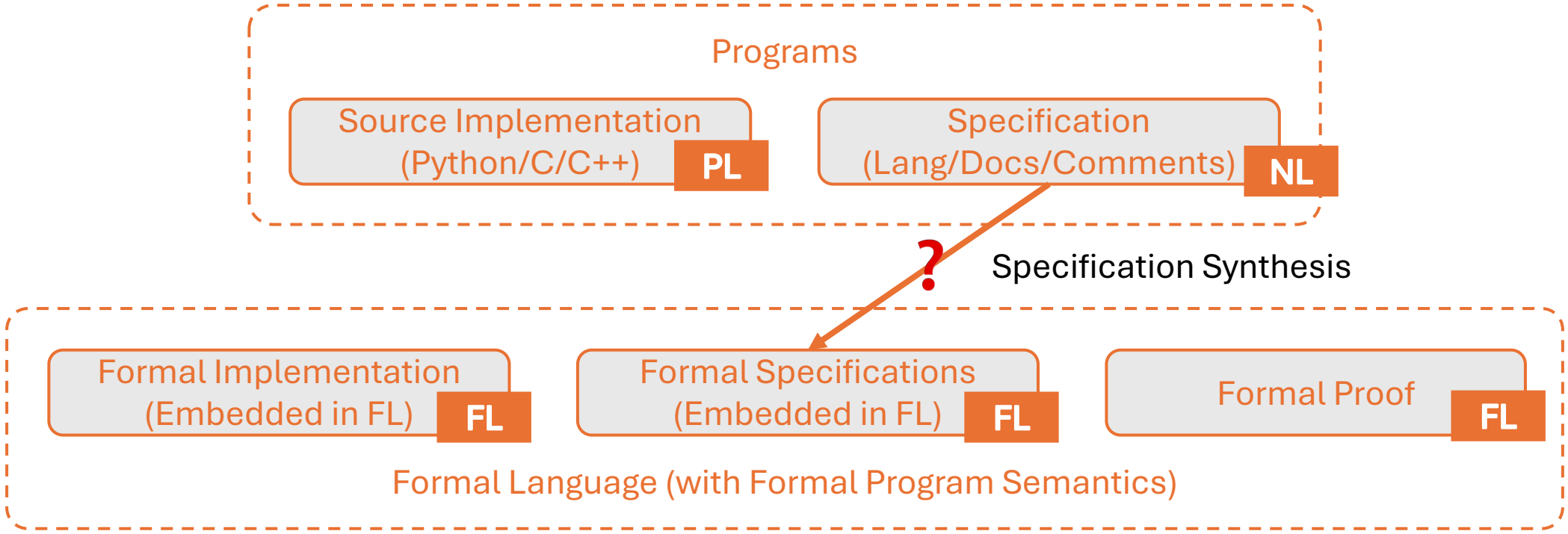


Natural Language (NL)

Programming Language (PL)

Formal Language (FL)

Formal Program Verification

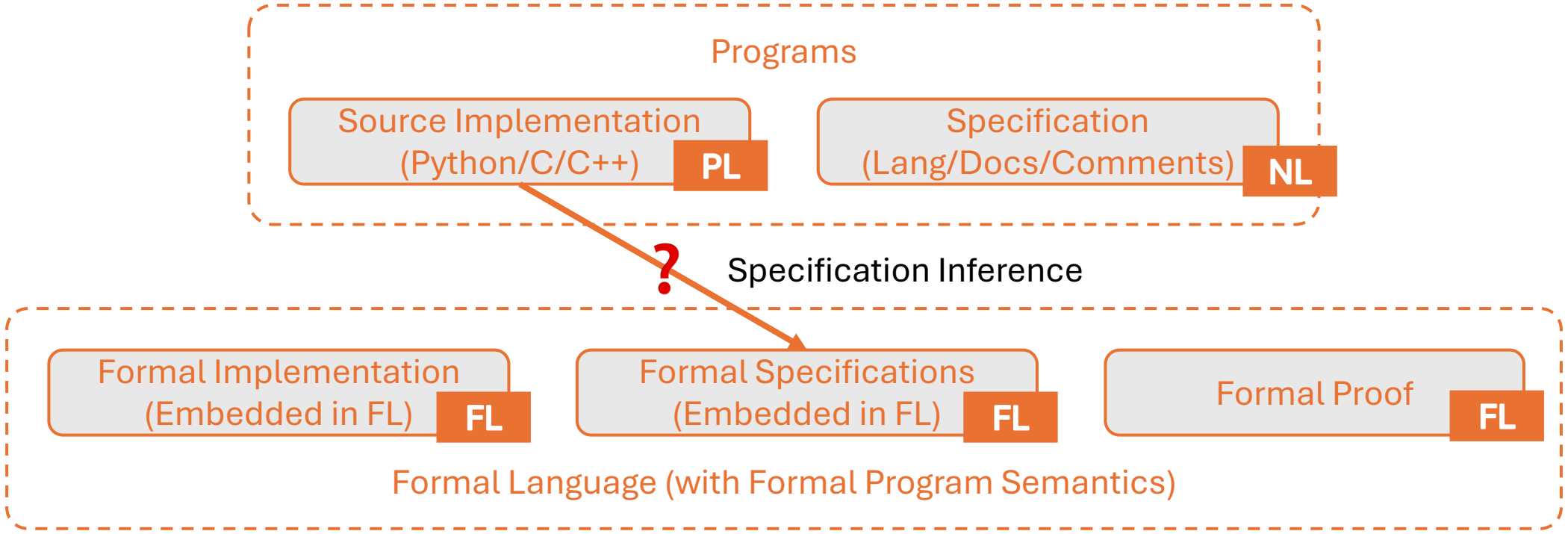


Natural Language (NL)

Programming Language (PL)

Formal Language (FL)

Formal Program Verification



Natural Language (NL)

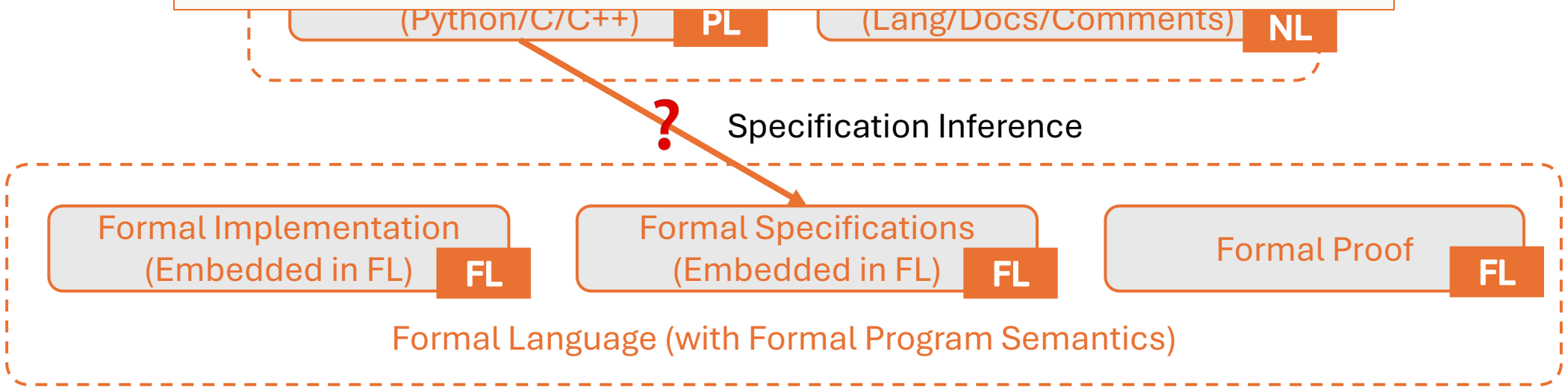
Programming Language (PL)

Formal Language (FL)

Forma

Turning tests into proofs

To prove that a piece of software is correct, we must first state what "correct" even means. Correctness is defined by the *specification* (spec). In some programs, there is a clear way to formally specify the program's intended behavior. For example, a PNG decoder takes a sequence of bytes and (following RFC 2083) decodes these bytes into the corresponding image. But for most programs, things aren't so straightforward. What would it mean for a text editor or a web browser to be "correct"? The difficulty of obtaining a comprehensive and unambiguous spec is a significant obstacle to verifying software at a larger scale.

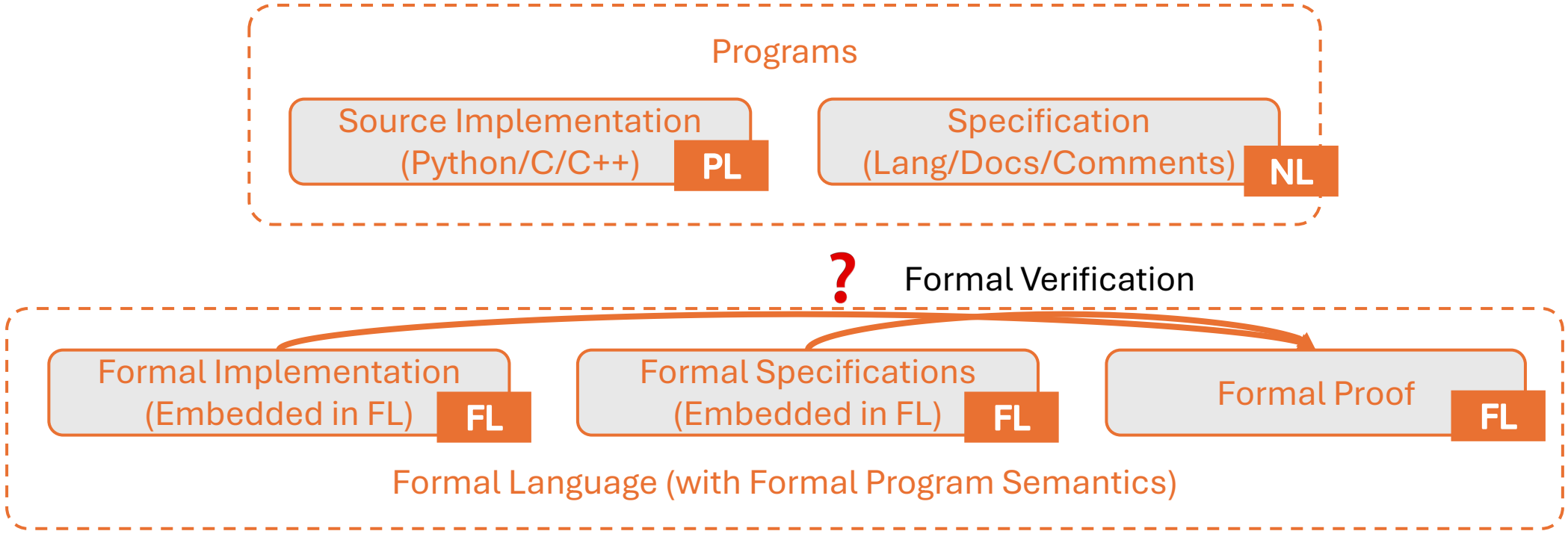


Natural Language (NL)

Programming Language (PL)

Formal Language (FL)

Formal Program Verification



Natural Language (NL)

Programming Language (PL)

Formal Language (FL)

Demo

- Last lecture: abs function shallowly embedded in Lean
- This lecture: abs/max/fac function deeply embedded in Lean
  - With a virtual machine: assembly interpreter
  - Formal specification on termination

ETAPS
INTERNATIONAL JOINT CONFERENCES ON THEORY AND PRACTICE OF SOFTWARE

TACAS

Competition on Software Verification (SV-COMP)

Motivation

Competition is a driving force for the invention of new methods, technologies, and tools. This web page describes the competition of software-verification tools, which will take place at TACAS.

There are several new and powerful software-verification tools around, but they are very difficult to compare. The reason is that so far no widely distributed benchmark suite of verification tasks was available and most concepts are only validated in research prototypes. This competition has changed this: Now there is an established set of verification tasks for comparing software verifiers, and the tools are publicized on the SV-COMP web site.

Only few projects aim at producing stable tools that can be used by people outside the respective development groups, and the development of such tools is not continuous. Also, PhD students and PostDocs do not adequately benefit from tool development because theoretical papers count more than papers that present technical contributions, like tool papers. Through its visibility, this competition wants to change this, showing off the latest implementation of the research results in our community, and give credits and benefits to researchers and students who spend considerable amounts of time developing verification algorithms and software packages.

ETAPS
INTERNATIONAL JOINT CONFERENCES ON THEORY AND PRACTICE OF SOFTWARE

TACAS

Competition on Software

Motivation

Competition is a driving force for the invention of new r
page describes the competition of software-verification

There are several new and powerful software-verificati
compare. The reason is that so far no widely distribute
available and most concepts are only validated in rese
this: Now there is an established set of verification task
tools are publicized on the SV-COMP web site.

Only few projects aim at producing stable tools that ca
development groups, and the development of such too
PostDocs do not adequately benefit from tool develop
than papers that present technical contributions, like to
competition wants to change this, showing off the lates
community, and give credits and benefits to researcher
amounts of time developing verification algorithms and

Competition Editions

2027
- Description

2026
- Description
- Verification
- Validation
- Report
- Reference

2025
- Description
- Verification
- Validation
- Report
- Reference

2024
- Description
- Verification
- Validation
- Report
- Reference

2023
- Description
- Verification
- Validation
- Report
- Reference

2022
- Description
- Verification
- Report
- Reference

2021
- Description
- Verification
- Report
- Reference

2020
- Description
- Verification
- Report
- Reference

2019
- Description
- Verification
- Report
- Reference

ETAPS
INTERNATIONAL JOINT CONFERENCES ON THEORY AND PRACTICE OF SOFTWARE

TACAS

(SV-COMP)

Competition Editions

2027

Description

2026

2025

2024

Description
Verification
Validation
Report
Reference

2021

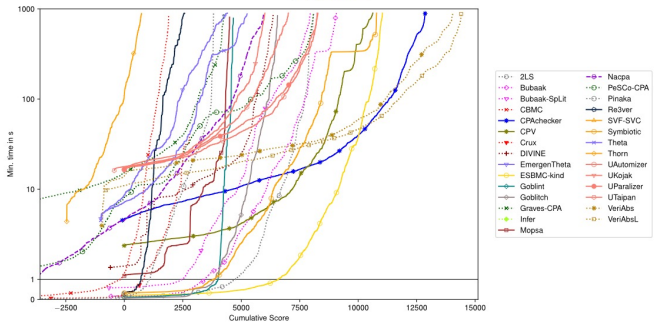
Description
Verification
Report
Reference

1. C Verification

Ranking by Category (with Score-Based Quantile Plots)

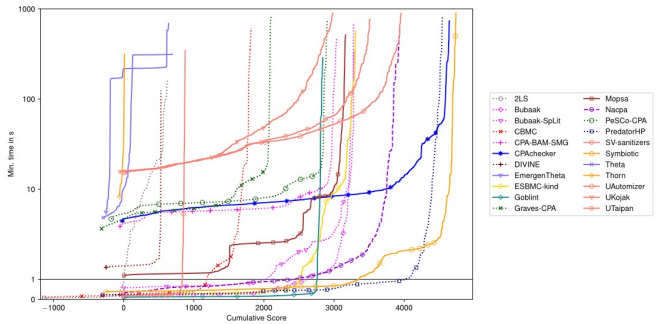
C.ReachSafety

1. CPAchecker
2. ESBMC-kind
3. Symbiotic



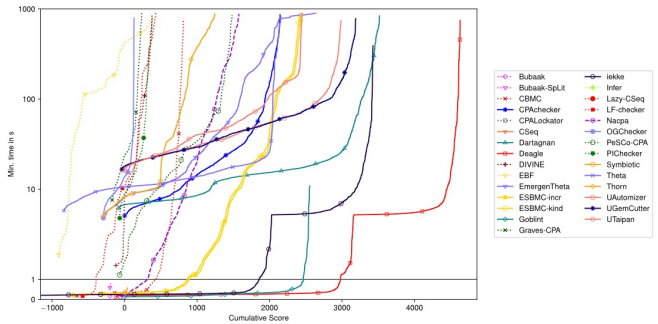
C.MemSafety

1. Symbiotic
2. CPAchecker
3. UAutomizer



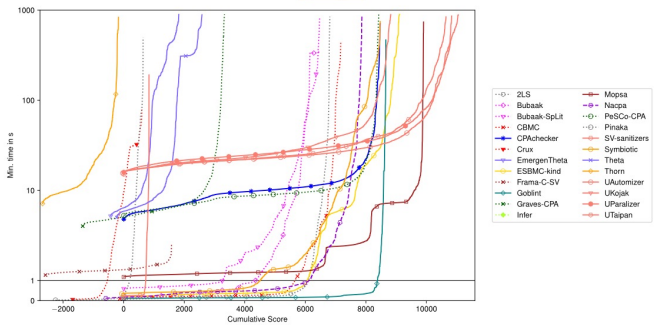
C.Concurrency

1. Deagle
2. Dartagnan
3. iekke



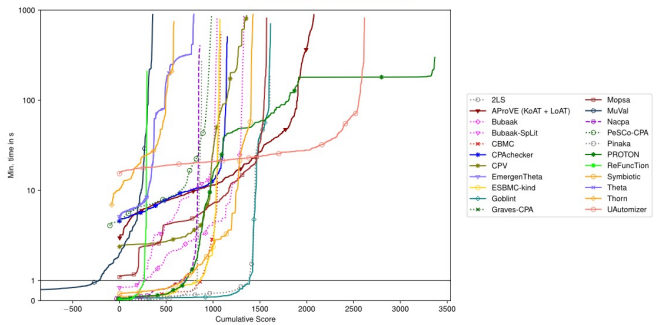
C.NoOverflows

1. UParalizer
2. UAutomizer
3. UTaipan



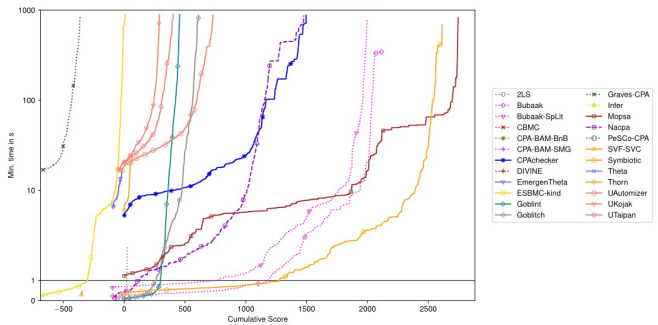
C.Termination

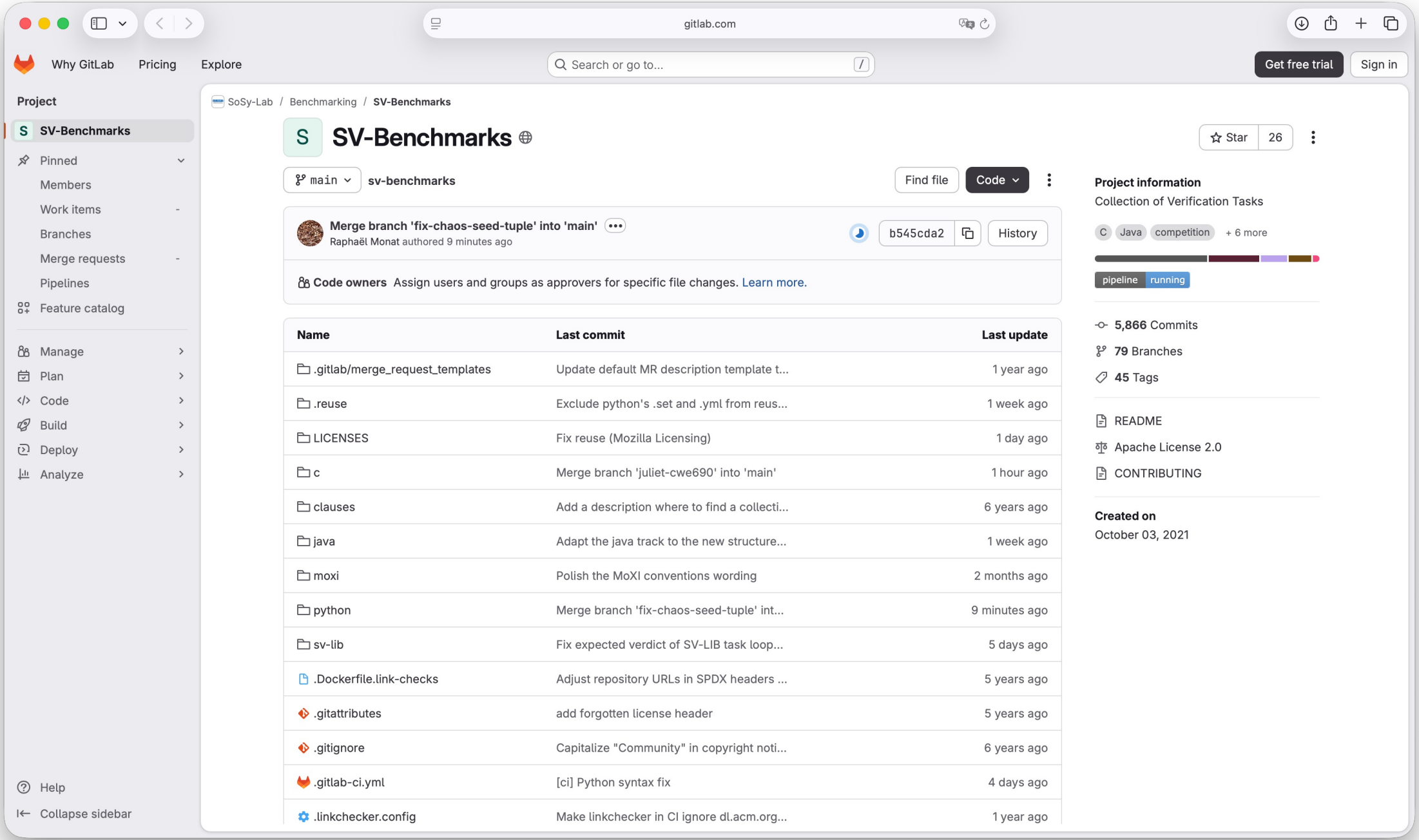
1. PROTON
2. UAutomizer
3. AProVE (KoAT + LoAT)

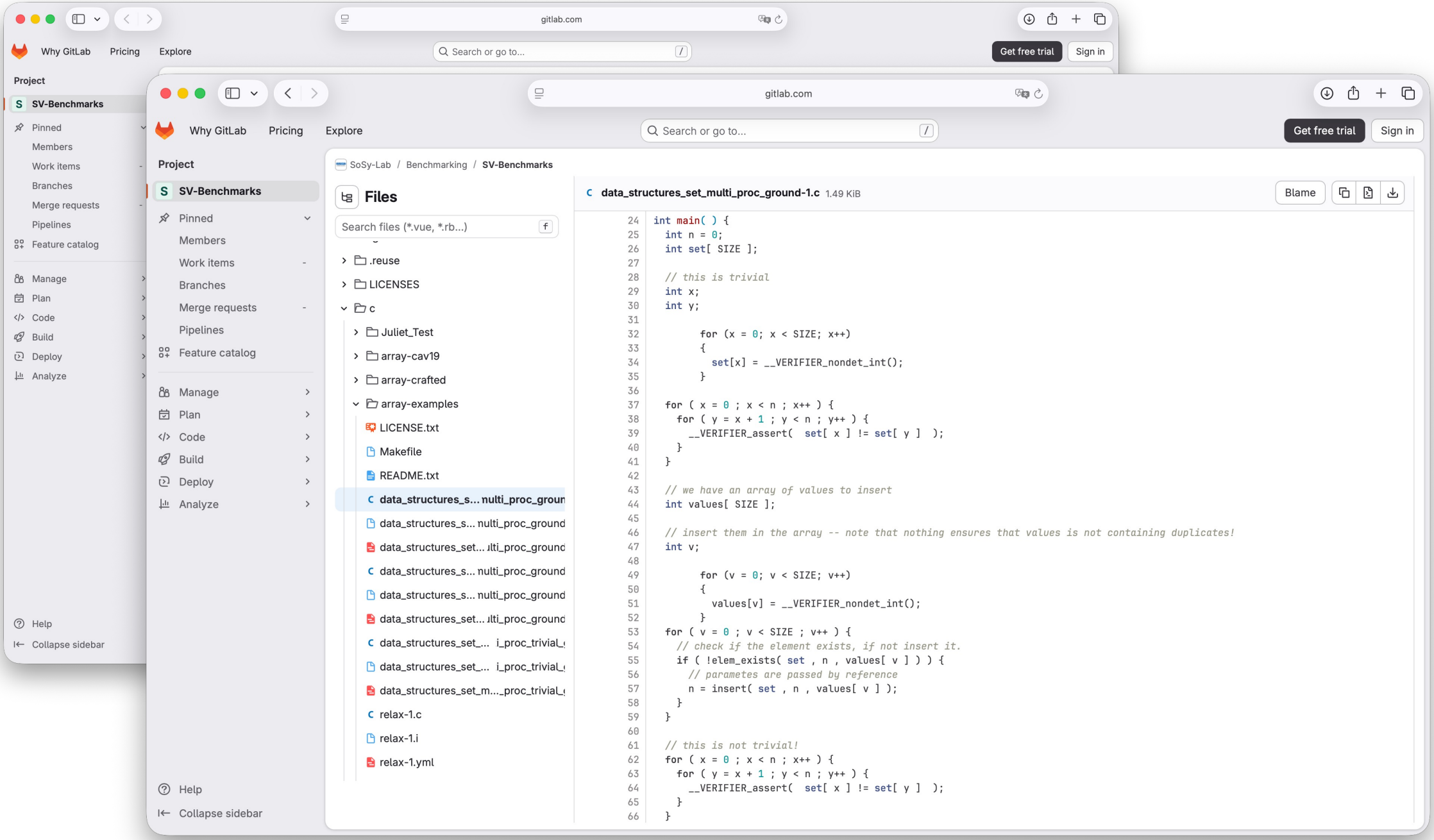


C.SoftwareSystems

1. Mopsa
2. Symbiotic
3. CPAchecker




gitlab.com
Why GitLab
Pricing
Explore
Search or go to...
Get free trial
Sign in
Project
SV-Benchmarks
Pinned
Members
Work items
Branches
Merge requests
Pipelines
Feature catalog
Manage
Plan
Code
Build
Deploy
Analyze
Help
Collapse sidebar
SoSy-Lab / Benchmarking / SV-Benchmarks
SV-Benchmarks
Star 26
main
sv-benchmarks
Find file
Code
Merge branch 'fix-chaos-seed-tuple' into 'main'
Raphaël Monat authored 9 minutes ago
b545cda2
History
Code owners Assign users and groups as approvers for specific file changes. Learn more.
Name
Last commit
Last update
.gitlab/merge_request_templates
Update default MR description template t...
1 year ago
.reuse
Exclude python's .set and .yml from reus...
1 week ago
LICENSES
Fix reuse (Mozilla Licensing)
1 day ago
c
Merge branch 'juliet-cwe690' into 'main'
1 hour ago
clauses
Add a description where to find a collecti...
6 years ago
java
Adapt the java track to the new structure...
1 week ago
moxi
Polish the MoXI conventions wording
2 months ago
python
Merge branch 'fix-chaos-seed-tuple' int...
9 minutes ago
sv-lib
Fix expected verdict of SV-LIB task loop...
5 days ago
.Dockerfile.link-checks
Adjust repository URLs in SPDX headers ...
5 years ago
.gitattributes
add forgotten license header
5 years ago
.gitignore
Capitalize "Community" in copyright noti...
6 years ago
.gitlab-ci.yml
[ci] Python syntax fix
4 days ago
.linkchecker.config
Make linkchecker in CI ignore dl.acm.org...
1 year ago
Project information
Collection of Verification Tasks
C
Java
competition
+ 6 more
pipeline
running
5,866 Commits
79 Branches
45 Tags
README
Apache License 2.0
CONTRIBUTING
Created on
October 03, 2021


gitlab.com
Why GitLab
Pricing
Explore
Search or go to...
Get free trial
Sign in
Project
SV-Benchmarks
Pinned
Members
Work items
Branches
Merge requests
Pipelines
Feature catalog
Manage
Plan
Code
Build
Deploy
Analyze
Help
Collapse sidebar
SoSy-Lab / Benchmarking / SV-Benchmarks
Files
Search files (*.vue, *.rb...)
.reuse
LICENSES
c
Juliet_Test
array-cav19
array-crafted
array-examples
LICENSE.txt
Makefile
README.txt
data_structures_s... nulti_proc_groun
data_structures_s... nulti_proc_ground
data_structures_set... ılti_proc_ground
data_structures_s... nulti_proc_ground
data_structures_s... nulti_proc_ground
data_structures_set... ılti_proc_ground
data_structures_set_... i_proc_trivial_
data_structures_set_... i_proc_trivial_
data_structures_set_m..._proc_trivial_
relax-1.c
relax-1.i
relax-1.yml
data_structures_set_multi_proc_ground-1.c 1.49 KiB
Blame
24 int main( ) {
25 int n = 0;
26 int set[SIZE];
27
28 // this is trivial
29 int x;
30 int y;
31
32 for (x = 0; x < SIZE; x++)
33 {
34 set[x] = __VERIFIER_nondet_int();
35 }
36
37 for (x = 0 ; x < n ; x++) {
38 for (y = x + 1 ; y < n ; y++) {
39 __VERIFIER_assert(set[x] != set[y]);
40 }
41 }
42
43 // we have an array of values to insert
44 int values[SIZE];
45
46 // insert them in the array -- note that nothing ensures that values is not containing duplicates!
47 int v;
48
49 for (v = 0; v < SIZE; v++)
50 {
51 values[v] = __VERIFIER_nondet_int();
52 }
53 for (v = 0 ; v < SIZE ; v++) {
54 // check if the element exists, if not insert it.
55 if (!elem_exists(set , n , values[v])) {
56 // parametes are passed by reference
57 n = insert(set , n , values[v]);
58 }
59 }
60
61 // this is not trivial!
62 for (x = 0 ; x < n ; x++) {
63 for (y = x + 1 ; y < n ; y++) {
64 __VERIFIER_assert(set[x] != set[y]);
65 }
66 }





PROTON 2.1: Synthesizing Ranking Functions via fine-tuned locally hosted LLM (Competition Contribution)

Diganta Mukhopadhyay[1], Ravindra Metta[1,2](✉), Hrishikesh Karmarkar[1], and Kumar Madhukar[3]

[1] TCS Research, Tata Consultancy Services, Pune, India
{diganta.m,r.metta,hrishi.karmarkar}@tcs.com
[2] School of CIT, Technical University of Munich, Munich, Germany
[3] Department of Computer Science, IIT Delhi, New Delhi, India
madhukar@cse.iitd.ac.in

Abstract. PROTON 2.1 presents (1) a new termination checking technique that uses a fine-tuned local LLM to synthesize ranking functions, and (2) support for multiple SAT solvers for non-termination checking.





PROTON 2.1: Synthesizing Ranking Functions
via fine-tune

Diganta
Hrishike

[1] TCS R
{diga
[2] School of CI
[3] Departmen

Abstract. PRO
nique that uses
and (2) support

1 PROTON 2.1: Ranking function synthesis via LLMs

The termination check in PROTON 1.1.2 [17] is based on loop unwinding assertions via CBMC [13]. In SV-COMP 2024 [11], this check failed on 123 benchmarks (e.g. Fig. 1), mainly as a large number of unwindings ($\geq 10^5$) are needed to satisfy the unwinding assertions in these benchmarks. To effectively check termination of such kinds of programs, we synthesize ranking functions by integrating a *small, locally hosted* LLM based on the Llama-3.2-1B-Instruct model (henceforth referred to as Llama-1B) [1] into the latest version of PROTON (2.1). This approach, henceforth referred to as LLM-T-Check, is similar to [16,15]. As out-of-the box Llama-1B failed to synthesize valid ranking functions even for trivial programs, we fine-tuned it to unlock its ability to guess candidate ranking functions (henceforth referred to as CRFs) and designed a check to validate the correctness of these guessed CRFs, as described below.



PROTON 2.1: Syn
via fine-tuned loca
C

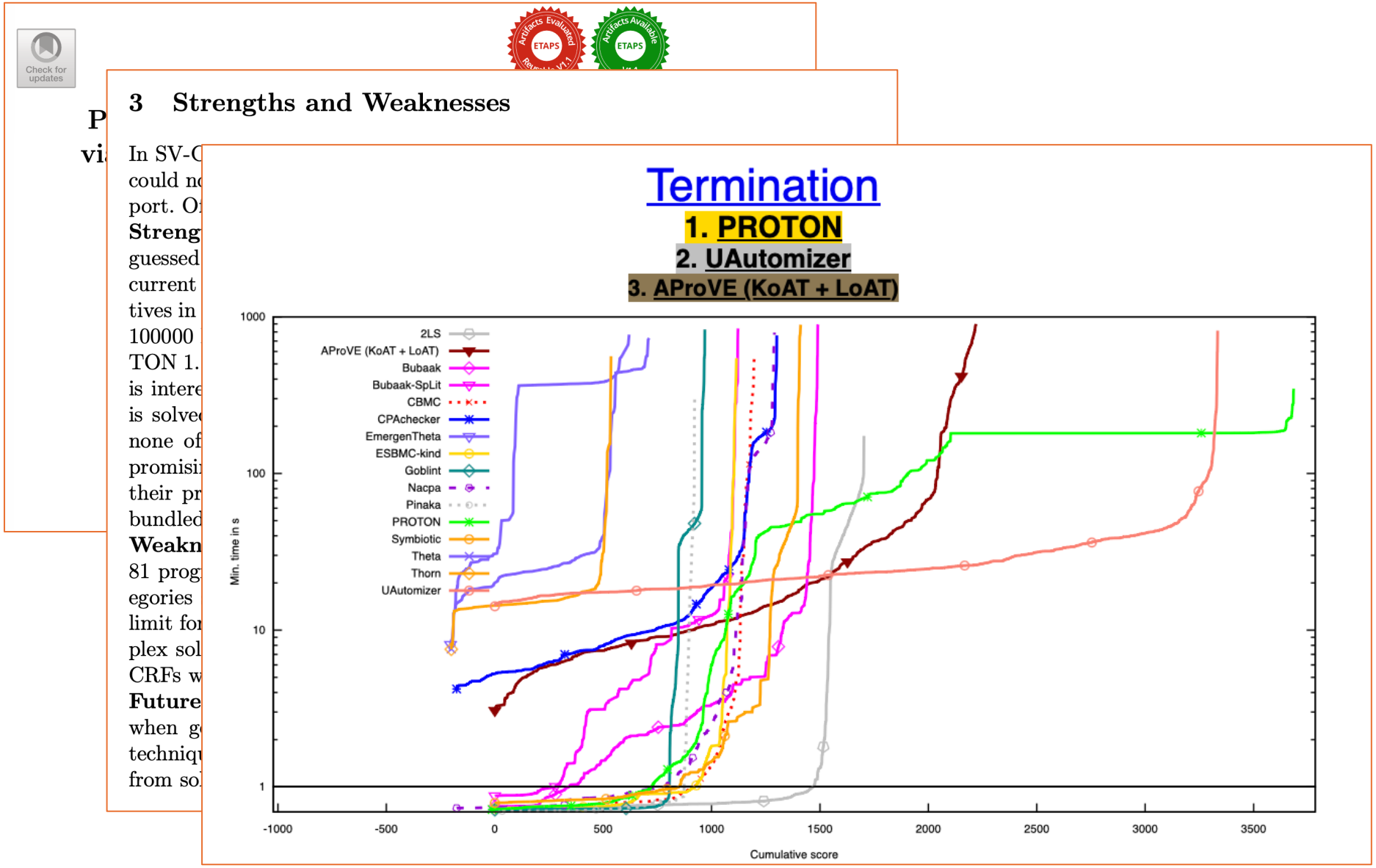
Diganta Mukhopa
Hrishikesh Karma

[1] TCS Research, T
{diganta.m,r.m
[2] School of CIT, Technic
[3] Department of Comp
madh

Abstract. PROTON 2.1
nique that uses a fine-tune
and (2) support for multip

3 Strengths and Weaknesses

In SV-COMP 2025, LLM-T-Check ran on 206 benchmarks that PROTON 1.1.2 could not solve. Of these, 23 are recursive, which our instrumenter does not support. Of the remaining 183, 102 were non-terminating and 81 were terminating.
Strengths: For the non-terminating benchmarks, incorrect ranking functions guessed by the LLM were rejected by the validation phase. Thus, even though our current validation check is unsound in theory, it is unlikely to produce false positives in practice. From the 81 terminating benchmarks, many require upwards of 100000 loop-unwinds for termination, and hence the termination check of PROTON 1.1.2 did not scale on these. LLM-T-Check is able to solve 14 of these 81. It is interesting to note that benchmark `array-examples/standard_sentinel-2` is solved only by LLM-T-Check with $(10^5 - \texttt{i})$ as the ranking function, while none of the other tools in SV-COMP 2025 are able to solve this. This is a promising first step in integrating locally-hosted LLMs, which in addition to their privacy and cost benefits, allows us to build offline modules that can be bundled into software verification tools.
Weaknesses: LLM-T-Check was unable to find ranking functions for 67 of the 81 programs. Of these, 45 are complex programs with multiple loops from categories such as `uthash` and `seq-mthreaded`. These programs exceed the token limit for the online hosted teacher Llama-70B, and hence will require more complex solutions to be able to generate CRFs. For the remaining 22, the guessed CRFs were invalidated during the checking phase.
Future work: Currently we are limited by the token limit of the online LLM when generating D. To overcome this, we are planning to use a fine-tuning technique that is independent of online hosted models, such as using feedback from solvers, circumventing the token limit.



3 Strengths and Weaknesses

COMPCERT

COMPILERS YOU CAN *FORMALLY* TRUST

ACM Software System Award 2021

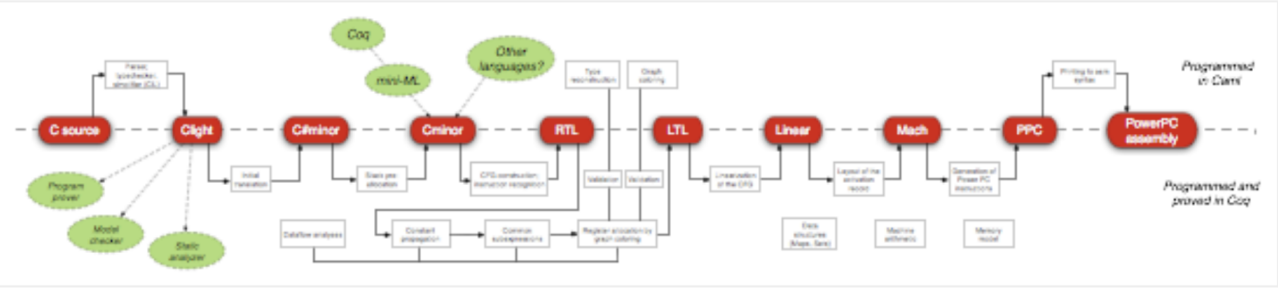
ACM SIGPLAN Programming Languages Software Award 2022

The CompCert project investigates the formal verification of realistic compilers usable for critical embedded software. Such verified compilers come with a mathematical, machine-checked proof that the generated executable code behaves exactly as prescribed by the semantics of the source program. By ruling out the possibility of compiler-introduced bugs, verified compilers strengthen the guarantees that can be obtained by applying formal methods to source programs.

The main result of the project is the CompCert C verified compiler, a high-assurance compiler for almost all of the C language (ISO C 2011), generating efficient code for the ARM, PowerPC, RISC-V and x86 processors.

Download CompCert C

Read the manual



MENU

- Home
- Partners
- Motivations
- Research
- The Compcert C compiler
- Downloads
- Publications

Formal Certification of a Compiler Back-end

or: Programming a Compiler with a Proof Assistant

Xavier Leroy

INRIA Rocquencourt

Xavier.Leroy@inria.fr

Formal Certification of a Compiler Back-end

***or*: Programming a Compiler with a Proof Assistant**

Xavier Leroy

Formal verification of a realistic compiler

Xavier Leroy
INRIA Paris-Rocquencourt
Domaine de Voluceau, B.P. 105, 78153 Le Chesnay, France
xavier.leroy@inria.fr

Formal Certification of a Compiler Back-end

Formal verification of a realistic compiler

Domain

Abstract

This paper reports on the development and formal verification (proof of semantic preservation) of CompCert, a compiler from Clight (a large subset of the C programming language) to PowerPC assembly code, using the Coq proof assistant both for programming the compiler and for proving its correctness. Such a verified compiler is useful in the context of critical software and its formal verification: the verification of the compiler guarantees that the safety properties proved on the source code hold for the executable compiled code as well.

Formal Certification of a Compiler Back-end

Formal verification of a realistic compiler

Abstract

This
tion
piler
guag
sista
its c
text
ficat
prov
code

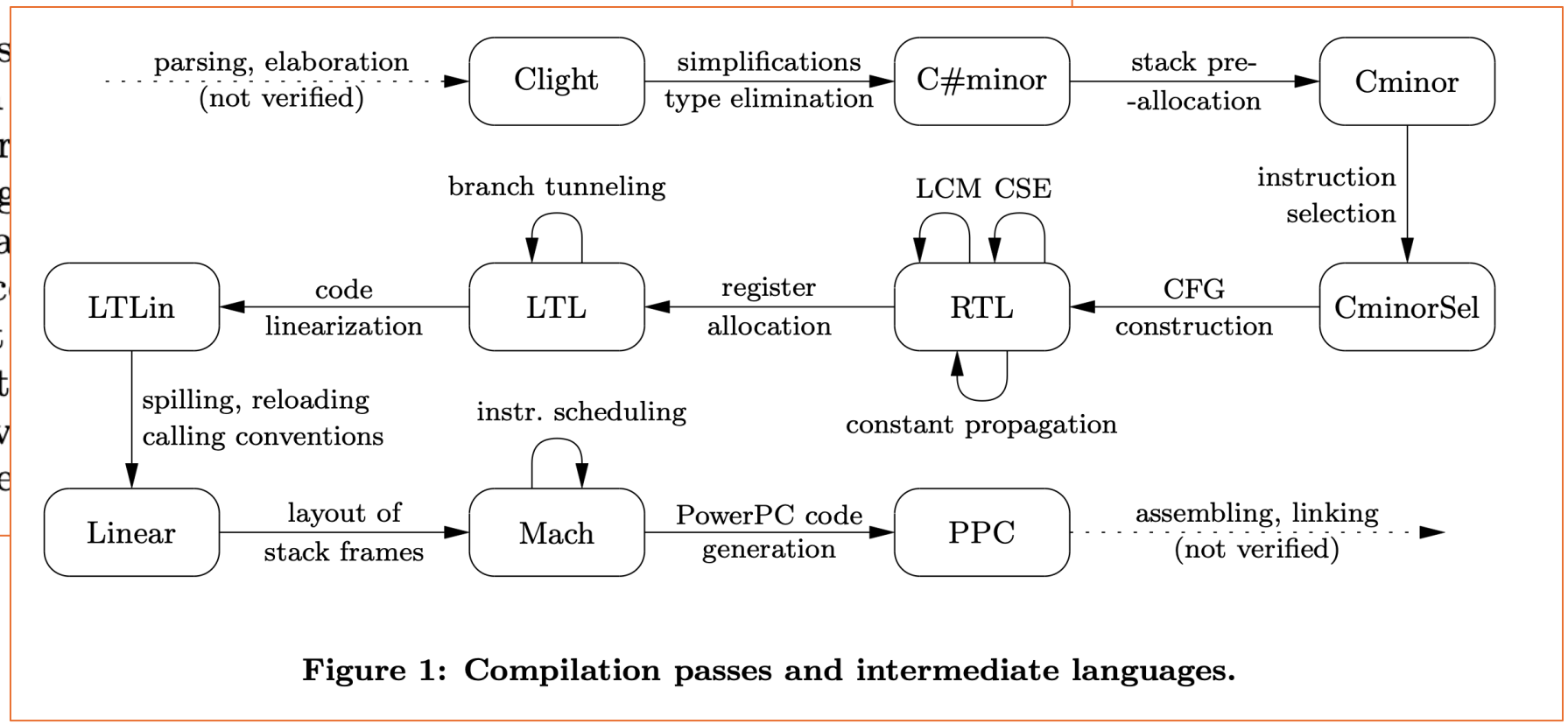


Figure 1: Compilation passes and intermediate languages.

seL4: Formal Verification of an OS Kernel

Gerwin Klein[1]
June Andronick[1,2], David Cock[1]
Rafal Kolanski[1,2], Michael Norri
[1] NICT

ABSTRACT

Complete formal verification is the only known way to guarantee that a system is free of programming errors.

We present our experience in performing the formal, machine-checked verification of the seL4 microkernel from an abstract specification down to its C implementation. We assume correctness of compiler, assembly code, and hardware, and we used a unique design approach that fuses formal and operating systems techniques. To our knowledge, this is the first formal proof of functional correctness of a complete, general-purpose operating-system kernel. Functional correctness means here that the implementation always strictly follows our high-level abstract specification of kernel behaviour. This encompasses traditional design and implementation safety properties such as the kernel will never crash, and it will never perform an unsafe operation. It also proves much more: we can predict precisely how the kernel will behave in every possible situation.

seL4, a third-generation microkernel of L4 provenance, comprises 8,700 lines of C code and 600 lines of assembler. Its performance is comparable to other high-performance L4 kernels.

A BENCHMARK FOR VERICODING:
FORMALLY VI

Sergiu Bursuc[1], **Theo**
Ionel Emilian Chiosa
Quinn Dougherty[1], **M**

We present and
formally verifie
which generates
benchmark con
Verus/Rust and
find vericoding
Dafny using off
significantly im
progress on pur
benchmark and

Table 1: Recent benchmarks for theorem proving, verification, vibe coding and vericoding. Verification and vericoding benchmarks are two orders of magnitude smaller than those for theorem proving and vibe coding. We list only benchmarks for Dafny, Verus and Lean. Notable benchmarks in other languages are SV-COMP (Beyer & Strejček, 2025) and SyGuS (Alur et al., 2018) in C and Java, and FVELER (Lin et al., 2024) and AFP (Archive of Formal Proofs) in Isabelle. In comparison, we have 12504 tasks of which 6174 are new or translated from other benchmarks.

Task	Benchmark	Language	Size
Thm proving	PutnamBench (Tsoukalas et al., 2024)	Lean, Isabelle, Coq	1709
Thm proving	FormalMATH (Yu et al., 2025)	Lean	5560
Thm proving	LeanDojo (Yang et al., 2023)	Lean	98734
Thm proving	LISA (Jiang et al., 2021)	Isabelle	183000
Verification	VeriBench (Miranda et al., 2025)	Lean	113
Verification	Verus-Bench (Yang et al., 2025)	Verus	150
Verification	Verified Cogen (JetBrains-Research, 2025)	Verus (among others)	223
Verification	VerifyThisBench (Deng et al., 2025)	Dafny, Why3, etc.	481
Verification	DafnyBench (Loughridge et al., 2025)	Dafny	782
Vibe coding	HumanEval (Chen et al., 2021)	Python	163
Vibe coding	FVAPPS (Dougherty & Mehta, 2025)	Lean	4715
Vibe coding	APPS (Hendrycks et al., 2021)	Python	10000
Vibe coding	CodeContests (Li et al., 2022)	Mixed	13610
Vibe coding	HumanEval-XL (Peng et al., 2024)	Mixed	22080
Vericoding	CLEVER (Thakur et al., 2025)	Lean	161
Vericoding	VERINA (Ye et al., 2025)	Lean	189

A BENCHMARK FOR VERICODING:

FOR

Serg
Ione
Quin

Table 1: Recent benchmarks for theorem proving, verification, vibe coding and vericoding. Verification and vericoding benchmarks are two orders of magnitude smaller than those for theorem proving and vibe coding. We list only benchmarks for Dafny, Verus and Lean. Notable benchmarks in other languag
and FV
have 12

Task	
Thm	
Thm	
Thm	
Thm	
Verif	
Verif	
Verif	
Verif	
Verif	
Vibe	
Vibe	
Vibe	
Vibe	
Vibe	
Veric	
Veric	

Lean (LB0000)

```
-- <vc-preamble>
def valid_bitstr (v : List Int) : Prop :=
  ∀i, i < v.length →(v[i]? = some 0 ∨
     v[i]? = some 1)
def str2int (v : List Int) : Nat :=
  match v with
  | [] => 0
  | x :: xs => x.toNat + 2 * str2int xs
-- </vc-preamble>
-- <vc-helpers> ... -- </vc-helpers>
-- <vc-definitions>
def add (v1 v2 : List Int) : List Int :=
  sorry
-- </vc-definitions>
-- <vc-theorems>
theorem add_spec (v1 v2 : List Int)
  (h1 : valid_bitstr v1) (h2 : valid_bitstr
       v2) :
  valid_bitstr (add v1 v2) ∧
  str2int (add v1 v2) = str2int v1 + str2int
        v2 :=
  by sorry
-- </vc-theorems>
-- <vc-postamble> ... -- </vc-postamble>
```

Verus (VB0000)

```
// <vc-preamble>
use vstd::prelude::*;
verus! {
spec fn valid_bitstr(v: Seq<i8>) -> bool
{ forall |i: int| 0 <= i < v.len() ==> (v[i] == 0 || v[i] ==
       1)}
spec fn str2int(v: Seq<i8>) -> int
   decreases v.len()
{ if v.len() == 0 { 0 } else { v[0] +
   2 * str2int(v.subrange(1, v.len() as int)) } }
// </vc-preamble>
// <vc-helpers> ... // </vc-helpers>
// <vc-spec>
fn add(v1: &Vec<i8>, v2: &Vec<i8>) -> (result: Vec<i8>)
   requires valid_bitstr(v1@) && valid_bitstr(v2@)
   ensures valid_bitstr(result@),
         str2int(result@) == str2int(v1@) + str2int(v2@)
// </vc-spec>
// <vc-code>
{ assume(false); unreached() }
// </vc-code>
// <vc-postamble>
} fn main() {}
// </vc-postamble>
```

Figure 1: Examples of vericoding tasks. We use *vc* tags for elements introduced in Section 3.1.

Table 3: Vericoding results

	NumPyS	NumPy3	DafnyBench	HumanEval	Verina	BigNum	VerifCogen	APPStest	Totals ↓
Dafny	58 tasks	430 tasks	443 tasks	162 tasks	157 tasks	62 tasks	172 tasks	677 tasks	**2161 tasks**
claude-opus-4.1	60.3%	64.9%	67.0%	71.6%	73.2%	14.5%	90.7%	66.6%	**67.5%**
gpt-5-mini	67.2%	56.3%	64.1%	71.6%	73.9%	25.8%	85.5%	71.6%	**66.9%**
gpt-5	65.5%	56.3%	63.9%	72.2%	76.4%	19.4%	86.6%	69.1%	**66.1%**
claude-sonnet-4	65.5%	69.3%	61.2%	72.8%	72.0%	8.1%	84.3%	60.3%	**64.6%**
gemini-2.5-pro	63.8%	60.2%	58.2%	69.1%	66.9%	6.5%	79.7%	40.8%	**55.0%**
grok-code	48.3%	47.0%	52.8%	57.4%	63.1%	3.2%	75.6%	52.9%	**53.0%**
glm-4.5	50.0%	41.4%	20.3%	50.0%	51.0%	4.8%	67.4%	48.7%	**42.0%**
gemini-2.5-flash	48.3%	34.4%	36.3%	42.6%	45.9%	0.0%	57.0%	36.9%	**38.2%**
deepseek-chat-v3.1	39.7%	43.0%	30.2%	45.1%	46.5%	3.2%	50.0%	30.6%	**36.2%**
model union	**75.9%**	**77.9%**	**71.9%**	**93.2%**	**87.3%**	**48.4%**	**95.9%**	**83.0%**	**82.2%**
Verus	58 tasks	581 tasks	443 tasks	161 tasks	156 tasks	62 tasks	172 tasks	536 tasks	**2166 tasks**
gpt-5	18.9%	47.5%	18.3%	15.5%	30.7%	3.2%	49.4%	26.5%	**30.9%**
claude-opus-4.1	18.9%	29.7%	19.2%	9.9%	26.2%	1.6%	63.4%	18.1%	**24.6%**
gemini-2.5-pro	34.5%	31.5%	19.7%	9.3%	25.6%	1.6%	52.9%	16.0%	**24.1%**
claude-sonnet-4	20.7%	22.0%	19.0%	5.6%	27.5%	1.6%	48.2%	13.4%	**19.9%**
glm-4.5	8.6%	23.0%	11.3%	5.6%	16.0%	0.0%	27.3%	16.8%	**16.6%**
gpt-5-mini	3.4%	21.3%	9.5%	6.2%	24.3%	0.0%	22.6%	19.0%	**16.5%**
grok-code	10.3%	22.9%	10.6%	3.1%	17.9%	1.6%	25.0%	13.8%	**15.5%**
gemini-2.5-flash	10.3%	16.3%	7.7%	1.2%	12.8%	0.0%	16.2%	5.2%	**9.8%**
deepseek-chat-v3.1	3.4%	6.9%	5.0%	1.8%	10.2%	0.0%	8.7%	6.5%	**6.1%**
model union	**37.9%**	**55.8%**	**34.8%**	**26.1%**	**46.8%**	**4.8%**	**77.9%**	**38.8%**	**44.3%**
Lean	59 tasks	603 tasks	440 tasks	161 tasks[1]	189 tasks	62 tasks	172 tasks	675 tasks	**2361 tasks**
gpt-5	45.8%	5.1%	32.0%	3.7%	14.3%	12.9%	34.9%	18.4%	**17.9%**
claude-sonnet-4	30.5%	0.7%	11.4%	1.2%	13.8%	1.6%	10.5%	24.0%	**11.9%**
gemini-2.5-pro	23.7%	0.3%	14.1%	3.1%	12.2%	1.6%	9.3%	19.9%	**10.9%**
claude-opus-4.1	20.3%	1.0%	11.8%	3.1%	15.3%	1.6%	10.5%	19.3%	**10.7%**
gpt-5-mini	15.3%	0.3%	15.0%	0.0%	1.6%	1.6%	4.1%	6.8%	**5.7%**
grok-code	10.2%	0.0%	9.1%	0.0%	1.1%	0.0%	4.7%	6.2%	**4.2%**
glm-4.5	6.8%	0.2%	2.5%	0.0%	0.5%	0.0%	1.7%	2.7%	**1.6%**
gemini-2.5-flash	0.0%	0.2%	0.5%	0.0%	0.0%	0.0%	0.6%	1.9%	**0.7%**
deepseek-chat-v3.1	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.1%	**0.0%**
model union	**52.5%**	**7.6%**	**34.1%**	**8.1%**	**25.4%**	**12.9%**	**44.2%**	**38.5%**	**26.8%**

[1] From the CLEVER benchmark

VERINA: BENCHMARKING VERIFIABLE CODE GENERATION

Zhe Ye[1], **Zhengxu Yan**[1], **Jingxuan He**[1], **Timothe Kasriel**[1], **Kaiyu Yang**[2*], **Dawn Song**[1]
[1]University of California, Berkeley, [2]Meta FAIR

ABSTRACT

Large language models (LLMs) are increasingly integrated in software development, but ensuring correctness in LLM-generated code remains challenging and often requires costly manual review. *Verifiable code generation*—jointly generating code, specifications, and proofs of code-specification alignment—offers a promising path to address this limitation and further unleash LLMs' benefits in coding. Yet, there exists a significant gap in evaluation: current benchmarks often focus on only individual components rather than providing a holistic evaluation framework of all tasks. In this paper, we introduce VERINA (Verifiable Code Generation Arena), a high-quality benchmark enabling a comprehensive and modular evaluation of code, specification, and proof generation as well as their compositions. VERINA consists of 189 manually curated coding tasks in Lean, with detailed problem descriptions, reference implementations, formal specifications, and extensive test suites. Our extensive evaluation of state-of-the-art LLMs reveals significant challenges in verifiable code generation, especially in proof generation, underscoring the need for improving LLM-based theorem provers in verification domains. The best model, OpenAI o3, achieves a 72.6% code correctness rate, 52.3% for specification soundness and completeness, and a mere 4.9% proof success rate (based on one trial per task). We hope VERINA will catalyze progress in verifiable code generation by providing a rigorous and comprehensive benchmark. We release our dataset on https://huggingface.co/datasets/sunblaze-ucb/verina and our evaluation code on https://github.com/sunblaze-ucb/verina.

VERINA: BENC
ATION

Zhe Ye[1], **Zhengxu Yan**[1],
[1]University of California,

Large language mo
ment, but ensuring
often requires cost
ating code, specifi
promising path to
coding. Yet, there
focus on only indi
framework of all ta
eration Arena), a h
evaluation of code
sitions. VERINA
detailed problem d
and extensive test s
significant challeng
underscoring the n
domains. The best
52.3% for specifica
cess rate (based on
verifiable code gen
We release our data
our evaluation code

Table 1: A comparison of VERINA with related prior works on LLMs for code generation and verification. We characterize whether each work supports the three foundational tasks for end-to-end verifiable code generation: CodeGen, SpecGen, ProofGen (Section 4.1). ● means fully supported, ◐ means partially supported, ○ means unsupported. If ProofGen is supported, we specify the proving style: automated theorem proving (ATP) or interactive theorem proving (ITP). For works supporting multiple tasks, we annotate if these tasks are supported in a modular and composable manner. Overall, VERINA offers more comprehensive and high-quality benchmarking compared to prior works.

		CodeGen	SpecGen	ProofGen	Proving Style	Compositionality	Language
Benchmarks	HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021)	●	○	○	–	–	Python
	Dafny-Synthesis (Misu et al., 2024)	●	◐	●	ATP	✗	Dafny
	DafnyBench (Loughridge et al., 2025)	○	○	●	ATP	–	Dafny
	miniCodeProps (Lohn & Welleck, 2024)	○	○	●	ITP	–	Lean
	FVAPPS (Dougherty & Mehta, 2025)	●	○	●	ITP	✗	Lean
Techniques	nl2postcond (Endres et al., 2024)	○	●	○	–	–	Python, Java
	Clover (Sun et al., 2024)	●	●	●	ATP	✗	Dafny
	AlphaVerus (Aggarwal et al., 2024)	●	○	●	ATP	✗	Rust
	AutoSpec (Wen et al., 2024)	○	●	●	ATP	✗	C/C++
	SpecGen (Ma et al., 2025)	○	●	●	ATP	✗	Java
	SAFE (Chen et al., 2025)	○	○	●	ATP	✗	Rust
	AutoVerus (Yang et al., 2025)	○	○	●	ATP	–	Rust
	Laurel (Mugnier et al., 2025)	○	○	●	ATP	–	Dafny
	Pei et al. (2023)	○	○	●	ATP	–	Java
	Baldur (First et al., 2023), Selene (Zhang et al., 2024)	○	○	●	ITP	–	Isabelle
	Rango (Thompson et al., 2025), PALM (Lu et al., 2024)	○	○	●	ITP	–	Coq
	VERINA	●	●	●	ITP	✓	Lean

VERINA

Zhe Ye[1], Z
[1]University

Table 1: A comparison of VERINA with related prior works on LLMs for code generation and verification. We characterize whether each work supports the three foundational tasks for end-to-end verifiable code generation: CodeGen, SpecGen, ProofGen (Section 4.1). ● means fully supported, ◐ means partially sup
style: automated th
multiple tasks, we a
VERINA offers mor

		CodeGen	SpecGen	ProofGen
Benchmarks	HumanEval (Chen et al.	●	○	○
	Dafny-Synthesis (Misu	○	○	●
	DafnyBench (Loughridg	○	○	●
	miniCodeProps (Lohn &	○	○	●
	FVAPPS (Dougherty &	●	●	○
Techniques	nl2postcond (Endres et a	○	●	○
	Clover (Sun et al., 2024	○	●	●
	AlphaVerus (Aggarwal	○	○	●
	AutoSpec (Wen et al., 2	○	●	●
	SpecGen (Ma et al., 202	○	●	●
	SAFE (Chen et al., 2025	○	●	●
	AutoVerus (Yang et al.,	○	○	●
	Laurel (Mugnier et al., 2	○	●	○
	Pei et al. (2023)	○	●	●
	Baldur (First et al., 2023	○	○	●
	Rango (Thompson et al.	○	○	●
	VERINA	●	●	●

Lar
mer
ofte
atin
pro
cod
focu
fran
erat
eval
sitio
deta
and
sign
und
domains. The best model, Ope
52.3% for specification soundne
cess rate (based on one trial per
verifiable code generation by pro
We release our dataset on https:/
our evaluation code on https://gi

```lean
-- Description of the coding problem in natural language
-- Remove an element from a given array of integers at a specified index. The resulting array should
-- contain all the original elements except for the one at the given index. Elements before the
-- removed element remain unchanged, and elements after it are shifted one position to the left.
-- Code implementation
def removeElement (s : Array Int) (k : Nat) (h_precond : removeElement_pre s k) : Array Int :=
  s.eraseIdx! k
-- Pre-condition
def removeElement_pre (s : Array Int) (k : Nat) : Prop :=
  k < s.size -- the index must be smaller than the array size
-- Post-condition
def removeElement_post (s : Array Int) (k : Nat) (res: Array Int) (h_precond : removeElement_pre s k)
    : Prop :=
  res.size = s.size - 1 ∧ -- Only one element is removed
  (∀ i, i < k → res[i]! = s[i]!) ∧ -- The elements before index k remain unchanged
  -- The elements after index k are shifted by one position
  (∀ i, i < res.size → i ≥ k → res[i]! = s[i + 1]!)
-- Proof (proof body omitted for brevity)
theorem removeElement_spec (s: Array Int) (k: Nat) (h_precond : removeElement_pre s k) :
  removeElement_post s k (removeElement s k h_precond) h_precond := by sorry
-- Test cases
(s : #[1, 2, 3, 4, 5]) (k : 2) (res : #[1, 2, 4, 5]) -- Positive test with valid inputs and output
-- Negative test cases
(s : #[1, 2, 3, 4, 5]) (k : 5) -- Inputs violate the pre-condition at Line 12
(s : #[1, 2, 3, 4, 5]) (k : 2) (res : #[1, 2, 4]) -- Output violates the post-condition at Line 16
(s : #[1, 2, 3, 4, 5]) (k : 2) (res : #[2, 2, 4, 5]) -- Output violates the post-condition at Line 17
(s : #[1, 2, 3, 4, 5]) (k : 2) (res : #[1, 2, 4, 4]) -- Output violates the post-condition at Line 18
```

Figure 1: An example instance of VERINA, consisting of a problem description, code implementation, specifications (pre-condition and post-condition), a proof (optional), and comprehensive test cases. Note that we select this instance for presentation purposes and VERINA contains more difficult ones.

VERINA

ATION

Zhe Ye[1], Z
[1]University

Table 1: A comparison of VERINA with related prior works on LLMs for code generation and verification. We characterize whether each work supports the three foundational tasks for end-to-end verifiable code generation: CodeGen, SpecGen, ProofGen (Section 4.1). ● means fully supported, ◐ means pa
style: au
multiple
VERINA

Benchmarks	Human
	Dafny-
	DafnyE
	miniCo
	FVAPP
Techniques	nl2post
	Clover
	AlphaV
	AutoSp
	SpecGe
	SAFE (
	AutoVe
	Laurel
	Pei et a
	Baldur
	Rango
	VERIN

```
1   -- Description of the coding problem in natural language
2   -- Remove an element from a given array of integers at a specified index. The resulting array should
3   -- contain all the original elements except for the one at the given index. Elements before the
4   -- rem
5   -- Cod
6   def re
7     s.er
8   -- Pre
9   def re
10    k <
11  -- Pos
12  def re
13      :
14    res.
15    (∀ i
16    -- T
17    (∀ i
18  -- Pro
19  theore
20    remo
21  -- Tes
22  (s : #
23  -- Neg
24  (s : #
25  (s : #
26  (s : #
27  (s : #
```

Figure 1: specificat
Note that

4.1 FOUNDATIONAL TASKS AND METRICS

As shown in Figure 2, all three foundational tasks include natural language descriptions and function signatures (Lines 7, 11, and 15 in Figure 1) as model inputs, which captures human intent and enforces consistent output formats, facilitating streamlined evaluation.

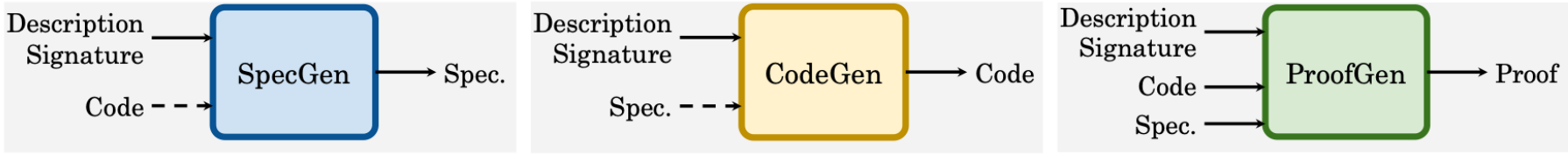


Figure 2: VERINA's three foundational tasks. Dashed arrows represent optional inputs.

Specification generation (SpecGen). Given a description, signature, and *optionally* code implementation, the model generates a formal specification. Next, we formally define the soundness and completeness relationships between the generated specification and the ground truth specification. Then, we describe our multi-stage evaluation pipeline to assess whether these relationships hold.